



对象的构造方法

北京理工大学计算机学院
金旭亮

一个值得注意的特殊方法.....

```
class MyTestClass {  
    public int Value;  
  
    public boolean equals(MyTestClass obj) {  
        return obj.Value == this.Value;  
    }  
  
    public MyTestClass(int initValue) {  
        Value = initValue;  
    }  
}
```



请总结一下，这个方法有哪些“与众不同之处”，你能列出几条？

类的构造方法



上页幻灯片所标出的方法，称为类的“构造方法（constructor）”，有时也习惯称为“构造函数”。



构造方法与类名相同，没有返回值。



当使用new关键字创建一个对象时，它的构造方法会被自动调用。



如果类没有定义构造函数，Java编译器在编译时会自动给它提供一个没有参数的“默认构造方法”。

动手动脑

以下代码为何无法通过编译？哪儿出错了？

```
public class Test {  
  
    public static void main(String[] args) {  
        Foo obj1=new Foo();  
    }  
}  
  
class Foo{  
    int value;  
    public Foo(int initValue) {  
        value=initValue;  
    }  
}
```



推出的结论



如果类提供了一个自定义的构造方法，将导致编译器为其默认提供的无参构造方法失效。

如果类的字段与构造方法的参数同名.....

```
class MyClass {  
    private String info;  
  
    public MyClass(String info) {  
        this.info = info;  
    }  
}
```



在构造函数中，如果函数参数与字段同名，那么，可以通过 **this** 关键字来区分引用的是字段还是方法参数。这种代码在实际开发中经常见到。

多构造函数

同一个类可以有多个构造函数,多个构造函数之间通过参数来区分,这是**方法重载 (method overload)** 的一个实例。

```
class Fruit {  
    int grams;  
    int calPerGram;  
    Fruit() {  
        this(55,10);  
    }  
    Fruit(int g,int c) {  
        grams = g;  
        calPerGram=c;  
    }  
}
```

同一个类的构造函数之间可以通过**this**关键字相互调用。



```
class RectangleBad {  
    private int x;  
    private int y;  
    private int width;  
    private int height;  
  
    public RectangleBad() {  
    }  
  
    public RectangleBad(int width, int height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    public RectangleBad(int x, int y,  
                        int width, int height) {  
        this.x = x;  
        this.y = y;  
        this.width = width;  
        this.height = height;  
    }  
}
```



```
class RectangleGood {  
    private int x;  
    private int y;  
    private int width;  
    private int height;  
  
    public RectangleGood() {  
        this( width: 0, height: 0);  
    }  
  
    public RectangleGood(int width, int height) {  
        this( x: 0, y: 0, width, height);  
    }  
  
    public RectangleGood(int x, int y,  
                        int width, int height) {  
        this.x = x;  
        this.y = y;  
        this.width = width;  
        this.height = height;  
    }  
}
```



类的初始化块



可以在类中使用“{”和“}”将语句包围起来，直接将其作为类的成员，称为“**类的初始化块**”。



类的这种“没有名字”的“成员”，多用于初始化类的字段。

```
public class Test {  
    public int value = 200;
```

```
    {  
        value=100;  
    }
```

```
}
```

类的初始化块

“自找麻烦”

如果一个类中既有初始化块，又有构造方法，同时还设定了字段的初始值，谁说了算？



```
class InitializeBlockClass{
{
    field=200;
}
public int field=100;
public InitializeBlockClass(int value){
    this.field=value;
}
public InitializeBlockClass(){

}
}
```



这是一个生造出来展示Java语法特性的示例类，在实际开发中不要这样写代码，应该尽量保证一个字段只初始化一次！

示例： InitializeBlockClass.java

进行试验

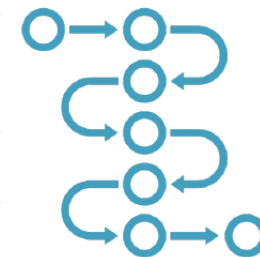


使用上页幻灯片中定义的类，以下代码输出结果是什么？

```
public static void main(String[] args) {  
  
    InitializeBlockClass obj=new InitializeBlockClass();  
    System.out.println(obj.field);//?  
  
    obj=new InitializeBlockClass(300);  
    System.out.println(obj.field);//?  
}
```

请依据代码的输出结果，自行总结Java字段初始化的规律（下页有答案，但还是请自己总结）

类字段的初始化顺序



1. 执行类成员定义时指定的默认值或类的初始化块，到底执行哪一个要看哪一个“排在前面”。
2. 执行类的构造函数。



类的初始化块不接收任何的参数，而且只要一创建类的对象，它们就会被执行。因此，适合于封装那些“**对象创建时必须执行的代码**”。

动手动脑



先给出一个论点：

当多个类之间有继承关系时，创建子类对象会导致父类初始化块的执行。

请自行编写示例代码确认以上论点是对还是错。



这里需要知道如何在Java中为两个类建立继承关系，相关内容在后面的课程中介绍，可以在学会后面课程的相关内容之后，再倒过来完成这个练习。

下一讲，介绍类的静态成员.....